Name: Fetalino Jayvee M.	**Date Performed: 12/02/2026**
Course/Section: CPE31S1	**Date Submitted: 12/02/2026**
Instructor: ENGR ROBIN VALENZUELA	**Semester and SY: 2025-2026 1st sem**

Activity 6: Targeting Specific Nodes and Managing Services

1. **Objectives:**

1.1 Individualize hosts

1.2 Apply tags in selecting plays to run

1.3 Managing Services from remote servers using playbooks

2. **Discussion**:

In this activity, we try to individualize hosts. For example, we don't want apache on all our servers, or maybe only one of our servers is a web server, or maybe we have different servers like database or file servers running different things on different categories of servers and that is what we are going to take a look at in this activity.

We also try to manage services that do not automatically run using the automations in playbook. For example, when we install web servers or httpd for CentOS, we notice that the service did not start automatically.

Requirement:

In this activity, you will need to create another Ubuntu VM and name it Server 3. Likewise, you need to activate the second adapter to a host-only adapter after the installations. Take note of the IP address of the Server 3. Make sure to use the command *ssh-copy-id* to copy the public key to Server 3. Verify if you can successfully SSH to Server 3.

Task 1: Targeting Specific Nodes

1.

```
---

- hosts: all
  become: true
  tasks:

  - name: install apache and php for Ubuntu servers
    apt:
      name:
        - apache2
        - libapache2-mod-php
      state: latest
      update_cache: yes
    when: ansible_distribution == "Ubuntu"

  - name: install apache and php for CentOS servers
    dnf:
      name:
        - httpd
        - php
      state: latest
    when: ansible_distribution == "CentOS"
```

2. Create a new playbook and named it site.yml. Follow the commands as shown in the image below. Make sure to save the file and exit.
3. Edit the inventory file. Remove the variables we put in our last activity and group according to the image shown below:

```
[web_servers]
192.168.56.120
192.168.56.121

[db_servers]
192.168.56.122


[file_servers]
192.168.56.123
```

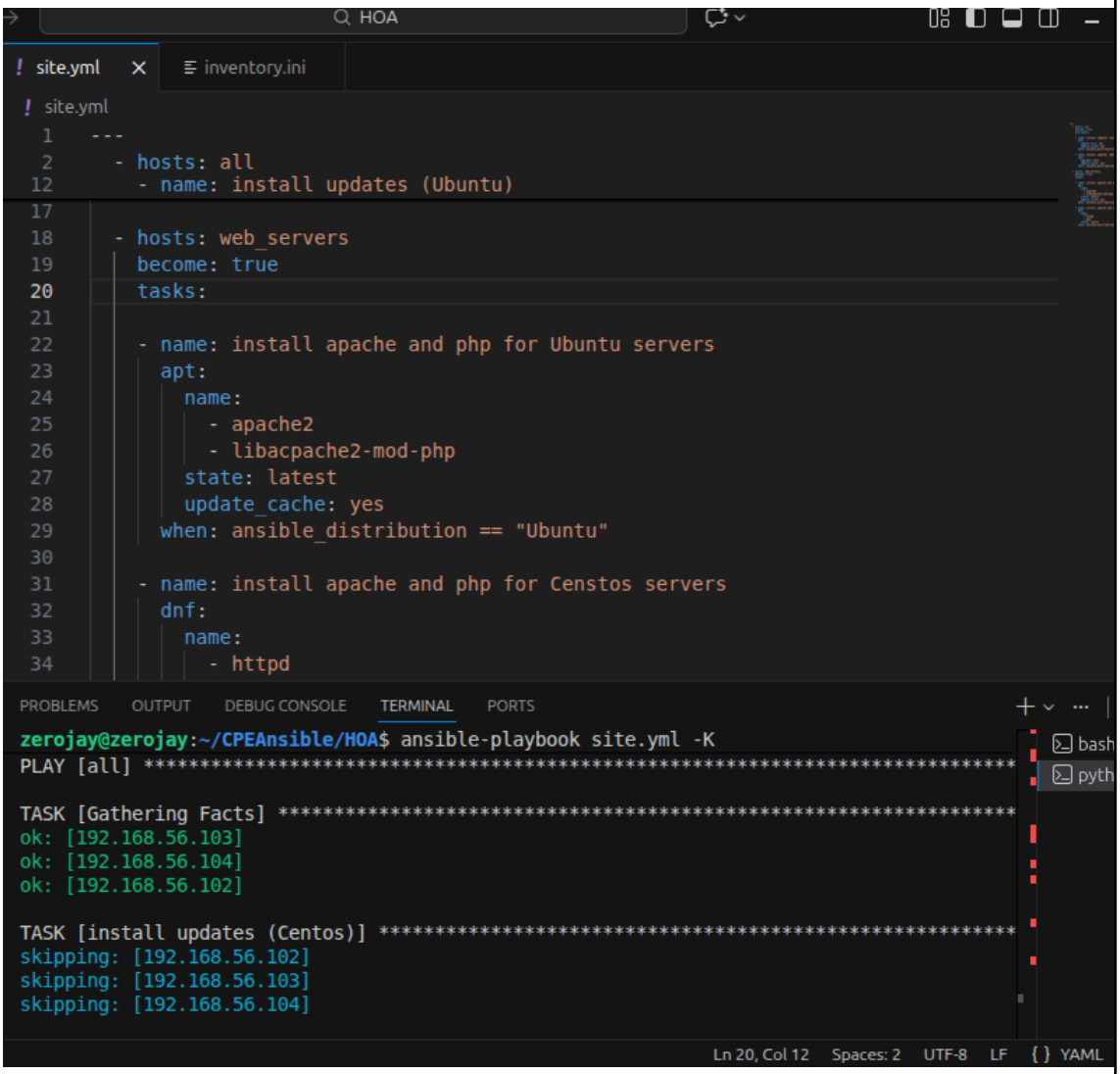
   Make sure to save the file and exit.

   Right now, we have created groups in our inventory file and put each server in its own group. In other cases, you can have a server be a member of multiple groups, for example you have a test server that is also a web server.
4. Edit the *site.yml* by following the image below:

```
---

- hosts: all
  become: true
  pre_tasks:

  - name: install updates (CentOS)
    dnf:
      update_only: yes
      update_cache: yes
    when: ansible_distribution == "CentOS"

  - name: install updates (Ubuntu)
    apt:
      upgrade: dist
      update_cache: yes
    when: ansible_distribution == "Ubuntu"


- hosts: web_servers
  become: true
  tasks:

  - name: install apache and php for Ubuntu servers
    apt:
      name:
        - apache2
        - libapache2-mod-php
      state: latest
    when: ansible_distribution == "Ubuntu"

  - name: install apache and php for CentOS servers
    dnf:
      name:
        - httpd
        - php
      state: latest
    when: ansible_distribution == "CentOS"
```

Make sure to save the file and exit.

The *pre-tasks* command tells the ansible to run it before any other thing. In the *pre-tasks*, CentOS will install updates while Ubuntu will upgrade its distribution package. This will run before running the second play, which is targeted at *web_servers*. In the second play, apache and php will be installed on both Ubuntu servers and CentOS servers.
Run the *site.yml* file and describe the result



.

When the first site.yml playbook is executed, the pre tasks run on all targeted servers. Ubuntu machines upgrade packages while CentOS installs updates. After that, Apache and PHP are installed on servers under the web_servers group, and the terminal shows completed tasks marked as ok or changed.

5. Let's try to edit again the *site.yml* file. This time, we are going to add plays targeting the other servers. This time we target the *db_servers* by adding it on the current *site.yml*. Below is an example: (Note add this at the end of the playbooks from task 1.3.

```
- hosts: db_servers
  become: true
  tasks:

  - name: install mariadb package (CentOS)
    yum:
      name: mariadb-server
      state: latest
    when: ansible_distribution == "CentOS"

  - name: "Mariadb- Restarting/Enabling"
    service:
      name: mariadb
      state: restarted
      enabled: true

  - name: install mariadb packege (Ubuntu)
    apt:
      name: mariadb-server
      state: latest
    when: ansible_distribution == "Ubuntu"
```

   Make sure to save the file and exit.

   Run the *site.yml* file and describe the result.
   After adding the db_servers play and running the updated site.yml, MariaDB is installed only on servers in the db_servers group. Other servers are not affected, and the output confirms successful execution on the targeted nodes.

6. Go to the remote server (Ubuntu) terminal that belongs to the db_servers group and check the status for mariadb installation using the command: *systemctl status mariadb.* Do this on the CentOS server also.

   Describe the output.

   On the Ubuntu server that belongs to the db_servers group, running the command systemctl status mariadb shows that MariaDB is installed and its service status is displayed. The output includes whether the service is active or inactive, the process ID, the time it started, and recent log messages. If the

installation was successful and the service is running, it will indicate an active running state.

7. Edit the *site.yml* again. This time we will append the code to configure installation on the *file_servers* group. We can add the following on our file.

```
- hosts: file_servers
  become: true
  tasks:

  - name: install samba package
    package:
      name: samba
      state: latest
```

Make sure to save the file and exit.

Run the *site.yml* file and describe the result.
When the site.yml file is updated to include the file_servers play and executed, file server packages are installed only on machines under the file_servers group. Web and database servers remain unchanged, and the output shows tasks applied only to file server hosts.

The testing of the *file_servers* is beyond the scope of this activity, and as well as our topics and objectives. However, in this activity we were able to show that we can target hosts or servers using grouping in ansible playbooks.

Task 2: Using Tags in running playbooks

In this task, our goal is to add metadata to our plays so that we can only run the plays that we want to run, and not all the plays in our playbook.

1. Edit the *site.yml* file. Add tags to the playbook. After the name, we can place the tags: *name_of_tag*. This is an arbitrary command, which means you can use any name for a tag.

```yaml
---

- hosts: all
  become: true
  pre_tasks:

  - name: install updates (CentOS)
    tags: always
    dnf:
      update_only: yes
      update_cache: yes
    when: ansible_distribution == "CentOS"

  - name: install updates (Ubuntu)
    tags: always
    apt:
      upgrade: dist
      update_cache: yes
    when: ansible_distribution == "Ubuntu"
```

```yaml
- hosts: web_servers
  become: true
  tasks:

  - name: install apache and php for Ubuntu servers
    tags: apache,apache2,ubuntu
    apt:
      name:
        - apache2
        - libapache2-mod-php
      state: latest
    when: ansible_distribution == "Ubuntu"

  - name: install apache and php for CentOS servers
    tags: apache,centos,httpd
    dnf:
      name:
        - httpd
        - php
      state: latest
    when: ansible_distribution == "CentOS"
```

```
- hosts: db_servers
  become: true
  tasks:

  - name: install mariadb package (CentOS)
    tags: centos, db,mariadb
    dnf:
      name: mariadb-server
      state: latest
    when: ansible_distribution == "CentOS"

  - name: "Mariadb- Restarting/Enabling"
    service:
      name: mariadb
      state: restarted
      enabled: true

  - name: install mariadb packege (Ubuntu)
    tags: db, mariadb,ubuntu
    apt:
      name: mariadb-server
      state: latest
    when: ansible_distribution == "Ubuntu"

- hosts: file_servers
  become: true
  tasks:

  - name: install samba package
    tags: samba
    package:
      name: samba
      state: latest
```

Make sure to save the file and exit.
Run the *site.yml* file and describe the result.
After adding tags and running site.yml, the playbook executes normally but each task is now labeled according to its tag. The output reflects the categorized tasks for easier identification.

2. On the local machine, try to issue the following commands and describe each result:

2.1 ansible-playbook --list-tags site.yml

2.2 ansible-playbook --tags centos --ask-become-pass site.yml

2.3 ansible-playbook --tags db --ask-become-pass site.yml

2.4 ansible-playbook --tags apache --ask-become-pass site.yml

2.5 *ansible-playbook --tags "apache,db" --ask-become-pass site.yml*

Executing ansible-playbook --tags centos --ask-become-pass site.yml runs only CentOS-related tasks while other tasks are skipped, and the output confirms execution limited to CentOS systems.

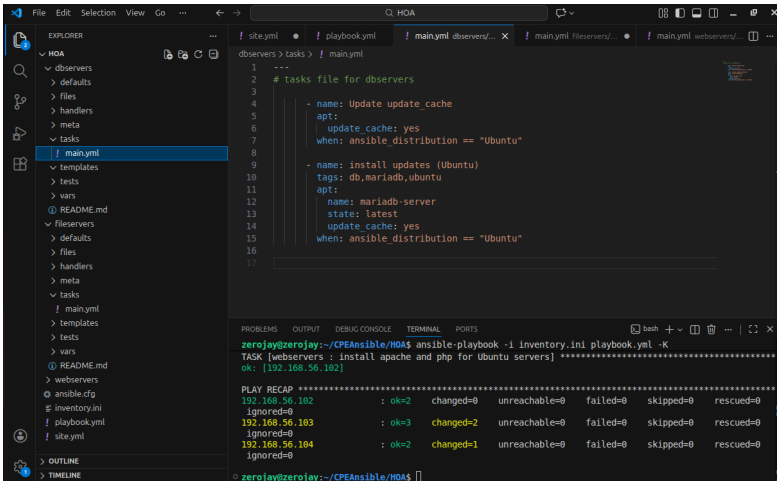
Running ansible-playbook --tags db --ask-become-pass site.yml performs only database-related tasks. MariaDB is installed or configured on db_servers while other services are skipped.

Using ansible-playbook --tags apache --ask-become-pass site.yml runs only Apache installation and configuration tasks, and database or file server tasks are not executed.

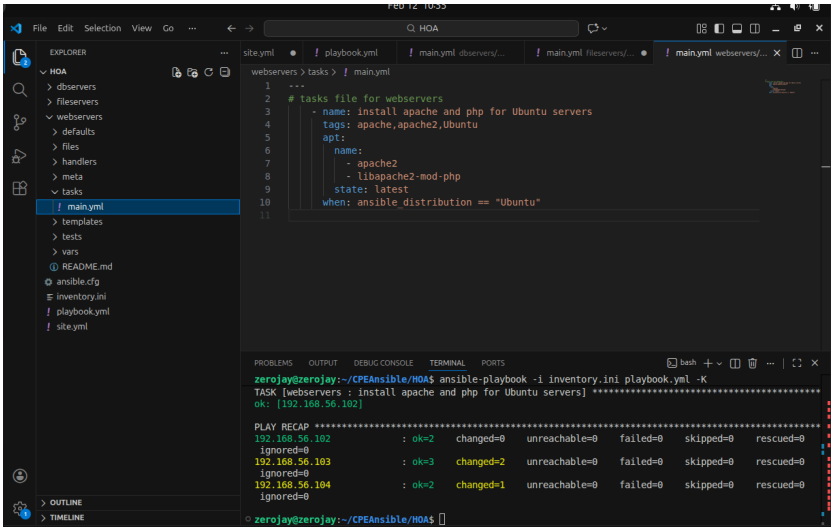
Executing ansible-playbook --tags apache,db --ask-become-pass site.yml runs both Apache and database tasks together, applying configurations to the appropriate servers while skipping unrelated tasks.

Supplementary Activity:

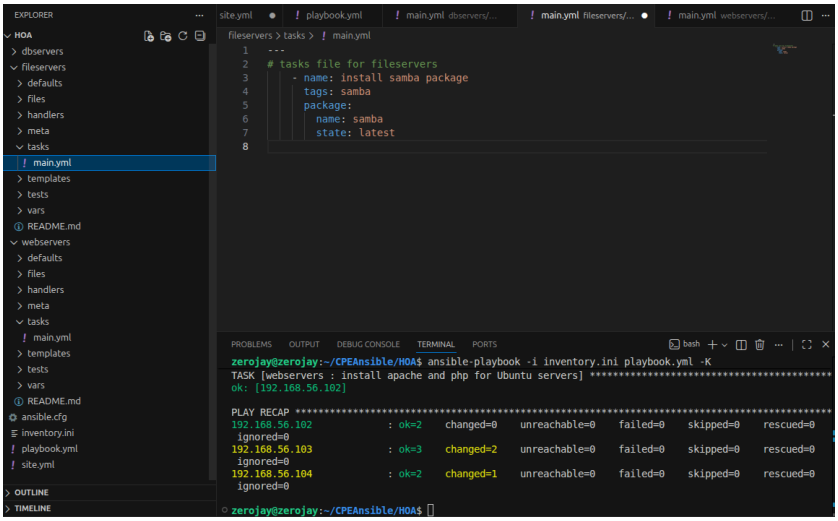
1. Creating a Role for Data Servers



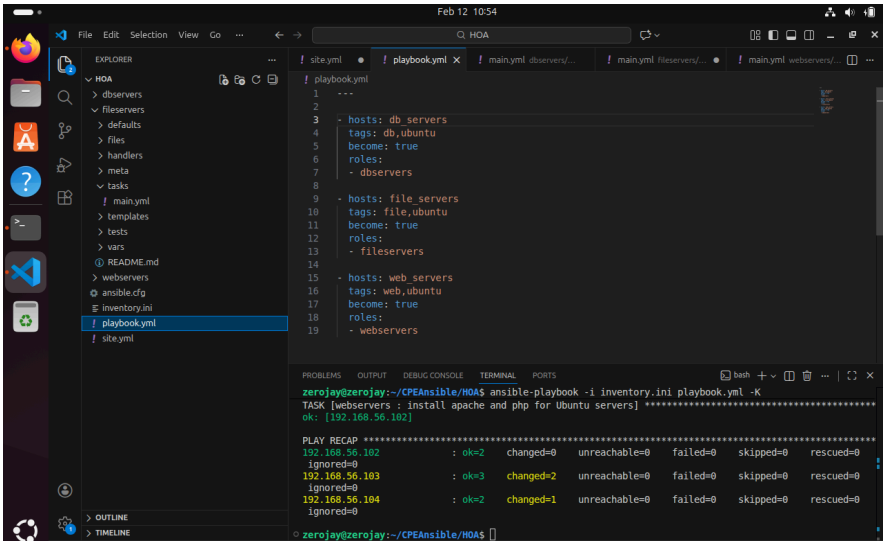
2. Creating a Role for Web Servers



3. Creating a Role for File Servers



4. Combining Multiple Roles in a Playbook



To orchestrate multiple roles in a single playbook for different server groups.

Add tags to each role to allow selective execution, e.g., tags: web for webserver, tags: db for database.

Task 3: Managing Services

1. Edit the file site.yml and add a play that will automatically start the httpd on CentOS server.

```
- name: install apache and php for CentOS servers
  tags: apache,centos,httpd
  dnf:
    name:
      - httpd
      - php
    state: latest
  when: ansible_distribution == "CentOS"

- name: start httpd (CentOS)
  tags: apache, centos,httpd
  service:
    name: httpd
    state: started
  when: ansible_distribution == "CentOS"
```

Figure 3.1.1
Make sure to save the file and exit.

You would also notice from our previous activity that we already created a module that runs a service.

```
- hosts: db_servers
  become: true
  tasks:

  - name: install mariadb package (CentOS)
    tags: centos, db,mariadb
    dnf:
      name: mariadb-server
      state: latest
    when: ansible_distribution == "CentOS"

  - name: "Mariadb- Restarting/Enabling"
    service:
      name: mariadb
      state: restarted
      enabled: true
```

Figure 3.1.2

This is because in CentOS, installed packages' services are not run automatically. Thus, we need to create the module to run it automatically.

2. To test it, before you run the saved playbook, go to the CentOS server and stop the currently running httpd using the command *sudo systemctl stop httpd*.

When prompted, enter the sudo password. After that, open the browser and enter the CentOS server's IP address. You should not be getting a display because we stopped the httpd service already.

3. Go to the local machine and this time, run the *site.yml* file. Then after running the file, go again to the CentOS server and enter its IP address on the browser. Describe the result.

   After stopping the httpd service on the CentOS server, accessing its IP address in a browser shows no webpage. After running the playbook again, the httpd service starts automatically and the default Apache webpage appears when the server IP is accessed.

   To automatically enable the service every time we run the playbook, use the command *enabled: true* similar to Figure 7.1.2 and save the playbook.

Reflections:

Answer the following:

1. What is the importance of putting our remote servers into groups?

Grouping remote servers allows tasks to be applied based on their roles, making management faster, organized, and easier to automate across multiple machines.

2. What is the importance of tags in playbooks?

Tags in playbooks help run only selected tasks instead of the whole playbook, saving time and making updates and troubleshooting more efficient.

3. Why do think some services need to be managed automatically in playbooks?

Managing services automatically ensures they are installed, started, and maintained consistently across servers, reducing manual work and preventing downtime.